

FastSET: Fast Sequential Retraining to Accelerate Real-Time Online Adaptation for Neural Receivers

Facheng Hu*, Yunzhe Li*, Hongzi Zhu*, and Xudong Wang[†]

*Shanghai Jiao Tong University, Email: {facheng_hu, yunzhe.li, hongzi}@sjtu.edu.cn

[†]Hong Kong University of Science and Technology (Guangzhou), Email: wxudong@ieee.org

Abstract—Neural receivers for wireless communication must adapt rapidly to time-varying channels to ensure reliable performance. However, frequent online adaptation incurs significant latency and computational overhead, limiting real-time deployment. Inspired by the modular design of conventional receivers, we propose FastSET, a fast sequential retraining strategy that decomposes the neural receiver into a neural estimator and a neural demapper, accelerating adaptation via a decoupled two-stage sequential update. An offline model search identifies an efficient estimator–demapper split that balances performance and complexity. During online operation, FastSET first updates the estimator with a few gradient steps, then refines the demapper while keeping the estimator frozen, mitigating the inherent moving-target problem and reducing total adaptation time. We evaluate FastSET on a frequency-domain OFDM receiver under 3GPP TDL channel models and ray-traced urban scenarios. Experimental results demonstrate that FastSET achieves 5.1–5.9× faster adaptation than end-to-end retraining, while limiting BER degradation to within 3% compared to the baseline across diverse dynamic environments.

Index Terms—Neural Receiver, Online Adaptation, OFDM, Channel Estimation, Deep Learning.

I. INTRODUCTION

Recent years have witnessed the rapid advancement of artificial intelligence (AI) technologies in wireless communication systems, particularly in domains such as physical-layer signal processing, network-layer resource management, and adaptive receiver design. AI is increasingly integrated throughout communication infrastructures, aligning with 6G’s vision of an end-to-end intelligent framework capable of supporting emerging ubiquitous services.

Deep neural networks (DNNs) have demonstrated transformative potential across diverse wireless tasks, including channel estimation [1]–[3], channel decoding [4], [5], beamforming optimization [6], [7], and radio resource allocation [8], [9]. In particular, neural network-based receivers (i.e., “neural receivers”) have emerged as promising candidates that outperform conventional designs [10]–[13]. By replacing discrete functional blocks with neural components or implementing full-stack learning, these receivers offer highly adaptive and resilient signal processing capabilities.

Compared to conventional paradigms, neural receivers provide three distinct advantages: 1) **Reduced Pilot Overhead**:

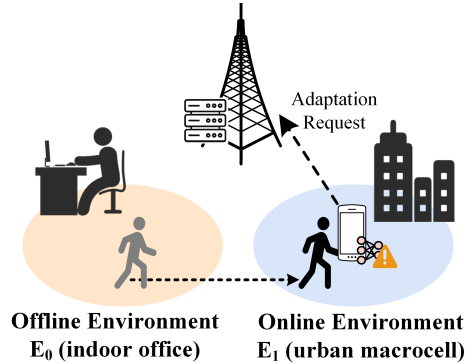


Fig. 1: Neural receivers experience accuracy degradation when users move across dynamically changing environments.

They significantly lower pilot requirements [14], [15], thereby enhancing spectral efficiency in high-frequency bands such as millimeter-wave and THz [16]. 2) **Implicit Channel Characterization**: Their data-driven nature enables the accurate capture of complex interference, non-linear hardware distortions, and quantization effects that are difficult to model analytically [17], [18]. 3) **Optimized Complexity-Performance Trade-offs**: They offer superior flexibility in balancing computational costs and predictive prowess, often surpassing the theoretical and practical limits of traditional receiver architectures [19].

Despite their potential, OFDM neural receivers face critical challenges in dynamic environments, as illustrated in Fig. 1. Conventional end-to-end retraining methods require hundreds of gradient iterations per adaptation cycle, resulting in computational delays that exceed practical coherence time constraints. Lightweight architectures that reduce adaptation overhead often compromise representational capacity, limiting performance in complex channel conditions. Consequently, practical adaptation frequencies are constrained to approximately 0.05–0.2 Hz in mobile scenarios [11], [12], severely affecting real-time operation.

The core challenge is that **online adaptation of neural receivers struggles to keep pace with rapid environmental changes**. An effective adaptation scheme must satisfy three requirements: rapid convergence to time-varying channel conditions, maintenance of near-optimal communication accuracy under diverse scenarios (including environmental transitions, multipath effects, and Doppler shifts), and computational efficiency compatible with constrained base station resources.

Corresponding author: Xudong Wang. The research was sponsored by the National Natural Science Foundation of China (NSFC) under Grant No. 62371292.

Prior works have approached this problem via lightweight architectures or meta-learning frameworks. Lightweight models reduce computation but often degrade accuracy under challenging channels. Meta-learning enables fast adaptation with a few gradient steps but requires extensive pretraining and is sensitive to distribution shifts. To the best of our knowledge, no existing scheme fully addresses real-time adaptation for neural receivers in practical wireless environments.

In this paper, we propose **FastSET**, a **Fast SEquential re-Training** strategy that significantly reduces the time overhead of online continual learning. The key idea is to partition the neural receiver into submodules and train them sequentially rather than updating the full model end-to-end. Given that training complexity typically scales super-linearly with model size, this divide-and-conquer approach substantially reduces adaptation time while maintaining near-optimal performance.

A second challenge is **how to optimally partition and train submodules to minimize computational overhead while preserving accuracy**. Improper partitioning can degrade performance or fail to achieve the desired acceleration. Furthermore, determining the optimal training schedule requires careful consideration for each submodule.

To address this, we decompose the receiver into a compact neural estimator and a larger neural demapper. The optimal architecture split is determined via an **Offline Model Search**. The estimator processes received signals and pilots to generate channel estimates, while the demapper uses these estimates alongside raw signals to produce log-likelihood ratios (LLRs), performing joint equalization and demodulation. The architectures closely follow the end-to-end model, ensuring that the total parameter count remains comparable. We define a metric called the **Performance–Efficiency Balance Score (PEBS)** and employ a grid search to identify the optimal estimator–demapper pair. This model is then used to initialize the online sequential adaptation process.

During **Online Sequential Adaptation**, the FastSET protocol is triggered when the measured bit error rate (BER) degrades beyond a threshold. First, the estimator is adapted with minimal gradient steps while the demapper remains frozen, capturing the current channel state rapidly. Then, the estimator is fixed and the demapper is fine-tuned using the improved channel estimates. This two-stage procedure ensures stable demapper training on reliable channel information, avoiding moving-target issues and enabling fast convergence.

We implement FastSET and conduct extensive simulations using NVIDIA’s Sionna library (3GPP TDL-A/B/C models) and ray-tracing simulators in urban environments (Paris and Munich). Results show that FastSET accelerates online adaptation by $5.1\text{--}5.9\times$ compared to end-to-end retraining while maintaining near-optimal BER performance across all tested scenarios. In urban ray-traced channels, FastSET retains 97% of the original model’s performance with $5.1\times$ acceleration.

The main contributions of this paper are:

- A divide-and-conquer architecture splitting the neural receiver into estimator and demapper modules to reduce adaptation time with minimal loss;

- A sequential training protocol with selective freezing, leveraging distinct convergence properties of estimation and demodulation components for fast adaptation;
- Extensive evaluations on 3GPP channel models and ray-traced urban scenarios, demonstrating FastSET’s efficacy in practical wireless environments.

II. RELATED WORK

Accelerating the real-time online adaptation of neural network-based receivers has become a significant research direction in modern wireless communication, particularly in scenarios involving high user mobility and limited computational resources. As neural receivers become increasingly prominent due to their ability to handle nonlinearities and complex channel effects, the demand for efficient adaptation strategies becomes critical. Existing literature addresses this challenge from three main perspectives: architectural simplification, meta-learning, and collaborative adaptation.

A. Model-Specific Neural Receiver Design

A prominent line of research explores the design of lightweight neural receiver architectures that inherently reduce the cost of online adaptation. Echo State Network (ESN)-based receivers are notable for their fixed recurrent structure and trainable linear output layer. The reservoir dynamics in ESNs encode temporal information without requiring back-propagation through time (BPTT), which significantly reduces computational overhead. Other lightweight architectures, such as shallow CNNs or linear receivers with adaptive heads, similarly trade off model complexity for adaptation speed.

However, these models often suffer from limited representational capacity, which constrains their ability to generalize to unseen channel conditions, especially in wideband or highly non-stationary environments. While fast to retrain, their performance often lags behind deeper, more expressive models. In particular, they lack robustness in multipath fading or interference-dominated scenarios. Moreover, tuning hyperparameters such as reservoir size or filter depth in shallow models becomes task-specific and brittle. Crucially, even these simplified architectures still require tens to hundreds of milliseconds per adaptation cycle—far exceeding the sub-millisecond coherence windows in high-mobility scenarios—thus failing to meet true real-time adaptation requirements where channel conditions evolve within single-digit millisecond durations.

B. Neural Receiver Adaptation

Meta-learning has been proposed as an elegant solution to the rapid adaptation problem. Methods such as Model-Agnostic Meta-Learning (MAML) [20], Reptile [21], and Context-based Adaptive Learning (CAVIA) allow models to learn a prior that can be quickly updated using only a few gradient steps. These methods have been applied to neural receivers in the literature, where MAML-like pretraining reduces adaptation latency while preserving performance across heterogeneous channel distributions.

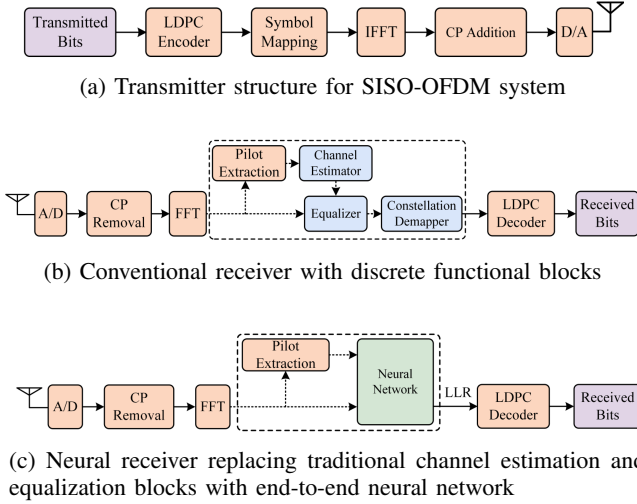


Fig. 2: Architecture comparison of transmitter, conventional receiver, and neural receiver in a SISO-OFDM system.

Bayesian extensions improve generalization under uncertainty by incorporating posterior distributions over model weights. However, most meta-learning methods require episodic training and careful task definition, which may be infeasible in continuous online learning environments. Additionally, the meta-trained prior is only effective within a known task distribution, and performance deteriorates when real-world deployment deviates from the training setup. Furthermore, meta-learning pipelines often add pretraining complexity and are sensitive to inner-loop learning rates and batch sizes. Most significantly, while reducing iteration counts, meta-learning still demands 50–200 gradient steps per adaptation cycle—at least an order of magnitude too slow for real-time adaptation in channels with coherence times below 10 ms—and introduces unacceptable demodulation gaps during weight updating.

III. BACKGROUND

A. Conventional Non-Learning Receiver

The operation of a conventional receiver follows a well-defined sequential process, as illustrated in Fig. 2. The process begins with the transmission of a discrete-time OFDM signal. After propagation through a multipath channel and OFDM demodulation at the receiver, the received signal in the frequency domain is given by:

$$Y[n, k] = H[n, k]X[n, k] + W[n, k], \quad (1)$$

where n denotes the OFDM symbol index, k denotes the subcarrier index, $X[n, k]$ is the transmitted symbol, $H[n, k]$ is the complex channel frequency response representing the wireless propagation effects, and $W[n, k] \sim \mathcal{CN}(0, \sigma^2)$ is additive white Gaussian noise with variance σ^2 .

The conventional receiver's signal processing chain comprises three discrete functional blocks, as shown in Fig. 2b:

Channel Estimation: The receiver estimates channel coefficients using known pilot symbols through dedicated estimation blocks. The Least Squares (LS) estimator computes:

$$\tilde{H}[n, k] = \frac{Y[n, k]}{X_p[n, k]}, \quad (n, k) \in \mathcal{P}, \quad (2)$$

where $\tilde{H}[n, k]$ denotes the estimated channel coefficient, $X_p[n, k]$ represents the known pilot symbol, and $\mathcal{P}$ is the set of pilot positions.

Equalization and Demodulation: Using the channel estimate, dedicated equalization blocks compensate for channel effects, followed by demodulation blocks that convert equalized symbols to soft-bit estimates (LLRs). The sequential nature of this processing chain ensures stable operation but lacks adaptability to changing channel conditions.

B. Neural Receiver Architecture

As depicted in Fig. 2c, the neural receiver replaces the discrete functional blocks of the conventional receiver with an integrated end-to-end neural network. This architecture learns a direct mapping from received signals to bit estimates, eliminating explicit signal processing stages and potentially offering superior performance in complex channel environments.

The input tensor combines the received signal and pilot information in a unified representation:

$$\mathbf{Z} = \text{Stack}(\Re\{\mathbf{Y}\}, \Im\{\mathbf{Y}\}, \Re\{\mathbf{P}\}, \Im\{\mathbf{P}\}) \in \mathbb{R}^{N_t \times N_f \times 4}, \quad (3)$$

where $\Re\{\cdot\}$ and $\Im\{\cdot\}$ denote the real and imaginary parts of a complex-valued matrix, respectively. $\mathbf{Y}$ denotes the received signal matrix, $\mathbf{P}$ represents the pilot matrix, and N_t, N_f are the time and frequency dimensions.

The end-to-end neural network implements the mapping:

$$\mathbf{L} = f_\theta(\mathbf{Z}), \quad (4)$$

where f_θ denotes the neural network parameterized by θ , and $\mathbf{L}$ represents the output LLR matrix. The network is trained to minimize the binary cross-entropy (BCE) loss:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [Q_i \log(\sigma(L_i)) + (1 - Q_i) \log(1 - \sigma(L_i))]. \quad (5)$$

While neural receivers demonstrate promising performance in controlled settings, their practical deployment faces challenges in dynamically changing environments where channel conditions evolve rapidly.

C. Online Adaptation Challenges

When deployed in changing environments, neural receivers require online adaptation to maintain performance. The adaptation process for conventional end-to-end architectures requires full-model retraining, which can be computationally expensive and time-consuming. The adaptation latency is characterized by:

$$\Delta t_A = \Delta t_R + \Delta t_T, \quad (6)$$

where Δt_R represents data collection time and Δt_T denotes model retraining time.

The fundamental challenge lies in the fact that conventional end-to-end adaptation approaches require simultaneous optimization of all parameters, leading to slow convergence and high computational overhead. This limitation becomes particularly critical in mobile scenarios where channel coherence times are short and computational resources are constrained.

These challenges motivate the need for efficient adaptation strategies that can maintain receiver performance while minimizing computational overhead, particularly for resource-constrained mobile devices operating in dynamic wireless environments.

IV. SYSTEM DESIGN

The FastSET framework addresses neural receiver adaptation challenges through a structured decomposition approach that separates architectural optimization from runtime adaptation, as illustrated in Fig. 3.

A. Offline Model Search (OMS)

The objective of Offline Model Search (OMS) is to identify an architectural configuration for the Neural Estimator and Neural Demapper that achieves a favorable balance between communication performance and training efficiency, while maintaining a representational capacity comparable to the original end-to-end (E2E) neural receiver. The architectural search is conducted under a strict complexity budget defined by the original E2E model, ensuring that the total number of trainable parameters of the decomposed receiver remains comparable to that of the baseline.

1) *Candidate Model Generation*: Let the original E2E model be characterized by a depth-width configuration $(d_{\text{E2E}}, w_{\text{E2E}})$, where depth corresponds to the number of residual blocks and width denotes the number of convolutional filters per layer. Based on this reference, OMS defines the architectural search space for the Neural Estimator by selecting its depth d_e and width w_e within bounded ranges:

$$d_e \in [d_{\min}, d_{\max}], \quad w_e \in [w_{\min}, w_{\max}], \quad (7)$$

where the bounds are chosen relative to the E2E configuration to preserve architectural similarity and sufficient representational capacity.

For each candidate pair (d_e, w_e) , the Neural Estimator is instantiated using a symmetric encoder-style residual architecture, in which intermediate layers gradually expand channel width before contracting toward the output layer. This design mirrors the structural characteristics of the E2E receiver and enables a fair comparison under matched complexity constraints.

After instantiating the Neural Estimator, its parameter count $\mathcal{P}_e(d_e, w_e)$ is computed explicitly. The remaining parameter budget allocated to the Neural Demapper is then obtained as:

$$\mathcal{P}_d = \mathcal{P}_{\text{E2E}} - \mathcal{P}_e(d_e, w_e), \quad (8)$$

where $\mathcal{P}_{\text{E2E}}$ denotes the total number of trainable parameters of the original E2E receiver.

The depth of the Neural Demapper, denoted by d_d , is determined proportionally according to the remaining parameter budget:

$$d_d = \left\lfloor \frac{\mathcal{P}_d}{\mathcal{P}_{\text{E2E}}} \cdot d_{\max} \right\rfloor. \quad (9)$$

Consequently, depth allocation is prioritized in the parameter distribution strategy. Given the demapper depth d_d , its width w_d is determined via a numerical bisection search such that the resulting parameter count $\mathcal{P}_d(d_d, w_d)$ closely matches the target $\mathcal{P}_d$. This procedure ensures:

$$\mathcal{P}_e(d_e, w_e) + \mathcal{P}_d(d_d, w_d) \approx \mathcal{P}_{\text{E2E}}, \quad (10)$$

thereby enforcing a consistent and fair complexity constraint across all candidate architectures.

2) *Model Assessment*: To enable a unified comparison across different architectural configurations, we adopt the *Performance-Efficiency Balance Score* (PEBS) as the evaluation metric during OMS. PEBS jointly accounts for communication performance and training efficiency, and is defined as:

$$\text{PEBS}(d_e, w_e) = \alpha \cdot \frac{\text{BER}_{\text{E2E}}}{\text{BER}_{\text{FastSET}}} + (1 - \alpha) \cdot \frac{T_{\text{E2E}}}{T_{\text{FastSET}}}, \quad (11)$$

where $\text{BER}_{\text{FastSET}}$ denotes the validation BER achieved by the candidate FastSET architecture after convergence, BER_{E2E} is the corresponding BER of the original E2E model, T_{FastSET} represents the total training time required for convergence, and T_{E2E} denotes the reference training cost of the E2E model. The weighting factor $\alpha \in [0, 1]$ controls the relative emphasis between performance preservation and training efficiency.

In this formulation, a larger PEBS value indicates a more favorable trade-off, corresponding to faster convergence and/or improved BER performance relative to the E2E baseline. In all experiments, α is empirically set to 0.8, reflecting a performance-oriented design choice while still accounting for training efficiency. PEBS is used exclusively for relative ranking during the offline OMS stage, rather than as an absolute performance metric.

Each candidate architectural configuration (d_e, w_e, d_d, w_d) is evaluated following the same procedure:

- 1) **Offline Training**: Execute the complete offline adaptation protocol, consisting of N_{bt}^e estimator update iterations followed by N_{bt}^d demapper update iterations.
- 2) **Performance Evaluation**: Measure the final validation BER $\text{BER}_{\text{FastSET}}$ after convergence.
- 3) **Efficiency Evaluation**: Record the total training cost T_{FastSET} required to reach the convergence criterion.
- 4) **PEBS Computation and Selection**: Compute PEBS for each candidate configuration. The resulting PEBS values form a heatmap over the architectural search space, and the configuration achieving the maximum PEBS is selected as the optimal model and passed to subsequent stages.

It is worth noting that while the generation and evaluation of candidate models are computationally intensive, these

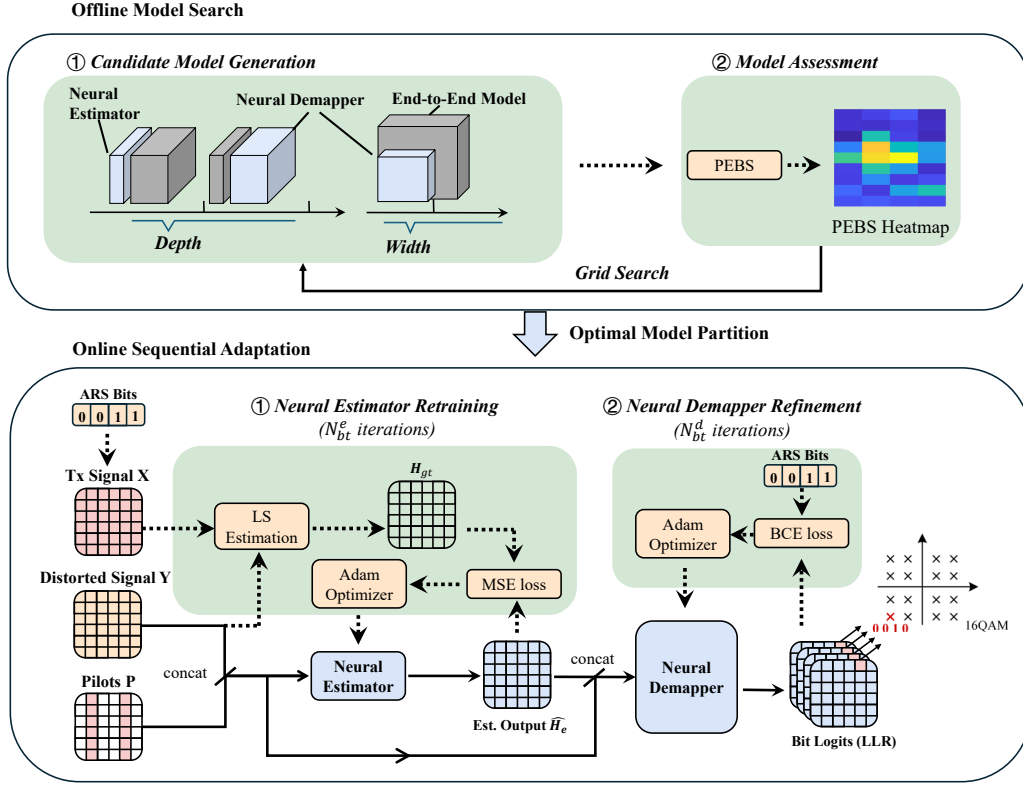


Fig. 3: Overview of the FastSET system, comprising two coordinated phases: Offline Model Search and Online Sequential Adaptation.

procedures are conducted offline as a one-time overhead. Consequently, they do not affect the efficiency or latency of the subsequent real-time online adaptation process.

B. Online Sequential Adaptation (OSA)

The Online Sequential Adaptation (OSA) implements a phased training strategy that respects the functional hierarchy of communication systems. As illustrated in Fig. 3, the process leverages three key inputs: the transmitted signal X (target bit stream), the received signal Y (channel-distorted observation of X), and pilot symbols P (known reference signals). The protocol proceeds through two sequential stages, ensuring that channel estimation and demodulation are optimized in a decoupled and interference-free manner.

1) *Neural Estimator Retraining*: The estimator retraining phase focuses on rapid adaptation to current channel conditions while maintaining gradient stability. During this N_{bt}^e -iteration phase, demapper parameters remain frozen to prevent interference and reduce computational overhead.

The estimator processes a composite input tensor constructed from received signals and pilot information:

$$\mathbf{Z} = \text{Stack}(\Re\{\mathbf{Y}\}, \Im\{\mathbf{Y}\}, \Re\{\mathbf{P}\}, \Im\{\mathbf{P}\}) \quad (12)$$

where $\mathbf{Y}$ represents the channel-distorted received signal and $\mathbf{P}$ contains the known pilot symbols.

The training objective minimizes the mean squared error (MSE) between the estimator's output and reference estimates:

$$\theta_e^* = \arg \min_{\theta_e} \mathbb{E}[\|f_{\theta_e}(\mathbf{Z}) - \mathbf{H}_{\text{ref}}\|_2^2]. \quad (13)$$

Reference estimates $\mathbf{H}_{\text{ref}}$ are obtained using conventional least squares (LS) estimation:

$$H_{\text{ref}}[n, k] = \frac{Y[n, k]}{P[n, k]}, \quad (n, k) \in \mathcal{P}, \quad (14)$$

where $\mathcal{P}$ denotes pilot positions in the time-frequency grid. This approach provides stable supervision signals while enabling rapid convergence within a few iterations (N_{bt}^e), which is crucial for tracking time-varying channels.

2) *Neural Demapper Refinement*: The demapper refinement phase optimizes symbol recovery using adapted channel estimates. During this N_{bt}^d -iteration phase, estimator parameters remain fixed to provide stable inputs while demapper parameters are optimized for bit-level recovery.

The demapper processes a comprehensive input tensor combining channel estimates and original signals:

$$\mathbf{I}_d = \text{Concat}(\hat{\mathbf{H}}_e, \Re\{\mathbf{Y}\}, \Im\{\mathbf{Y}\}, \Re\{\mathbf{P}\}, \Im\{\mathbf{P}\}), \quad (15)$$

where $\hat{\mathbf{H}}_e = f_{\theta_e^*}(\mathbf{Z})$ is the channel estimate from the frozen estimator.

Training employs binary cross-entropy (BCE) loss to optimize recovery accuracy. The input design provides complementary information: channel estimates capture current

TABLE I: Simulation Parameters for Performance Evaluation

Parameter	Value
Modulation scheme	16-QAM
OFDM symbols per frame (N_t)	14
Subcarriers per symbol (N_f)	72
Pilot spacing pattern	Every 6 subcarriers
Adaptation reference signals (ARS)	$N_q = 20$ blocks
Training batch size	8
Optimization algorithm	Adam
Learning rate	10^{-3}
3GPP channel models	TDL-A, TDL-B, TDL-C
SNR range	0–20 dB
UE velocity range	1–10 m/s
Number of random seeds	100
Adaptation trigger threshold (BER)	10^{-3}

propagation conditions, while raw signals offer context for handling estimation uncertainties.

V. EVALUATION

A. Simulation Setup

We conduct a comprehensive evaluation of FastSET under various channel conditions to validate its effectiveness in neural receiver adaptation. The experimental setup is designed to reflect practical deployment scenarios in 5G NR systems, focusing on realistic channel models and performance metrics.

1) *System Configuration*: Our experimental framework implements a SISO-OFDM downlink system compliant with 5G NR specifications. The system is developed using PyTorch 2.0 with CUDA acceleration on NVIDIA RTX 3090 GPUs, ensuring efficient training and inference. The baseband processing chain incorporates standard 5G NR physical-layer components, including LDPC channel coding (base graph 2, rate 1/2), cyclic-prefix insertion, and 3GPP-compliant DMRS pilot patterns.

The simulation parameters, summarized in Table I, are carefully selected to balance computational efficiency and practical relevance. We employ a 16-QAM modulation scheme across 72 active subcarriers with 14 OFDM symbols per frame, corresponding to one 5G NR slot duration of 1 ms at 15 kHz subcarrier spacing. Pilot signals are inserted every 6 subcarriers using comb-type pilot patterns consistent with 3GPP DMRS configuration type 1.

We consider both static and time-varying frequency-selective fading channels using 3GPP TR 38.901 TDL models (TDL-A, TDL-B, TDL-C) with exponential delay profiles. The channel evolution follows Jakes' Doppler spectrum with UE velocities ranging from 3 km/h (pedestrian scenarios) to 30 km/h (urban vehicular scenarios). For each channel realization, we generate correlated channel taps using the method of exact Doppler spread (MEDS) to ensure accurate temporal correlation properties. The channel is assumed to remain constant within each OFDM symbol but evolves across symbols according to the specified Doppler frequency.

The baseline neural receiver employs a ResNet-inspired architecture with an encoder–decoder structure. The encoder consists of four convolutional layers with kernel sizes [5, 3,

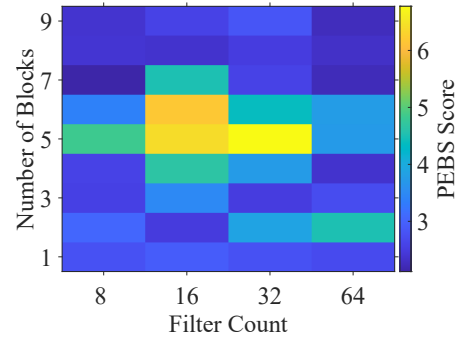


Fig. 4: PEBS heatmap over different combinations of network depth and filter width. A larger PEBS indicates a more favorable balance between BER performance and training efficiency.

3, 3] and feature dimensions [64, 128, 256, 512], using ReLU activation and layer normalization. The decoder employs symmetric transposed convolutions with skip connections to preserve spatial information.

For FastSET implementation, we partition the architecture after the second encoder layer: the estimator comprises the first two convolutional layers, while the demapper includes the remaining encoder layers and the complete decoder. This partitioning strategy ensures a balanced computational load between the two adaptation stages while maintaining sufficient representational capacity for both channel estimation and symbol demodulation tasks.

All results are averaged over 100 independent trials with different random seeds to ensure statistical significance. We report 95% confidence intervals where appropriate to quantify measurement uncertainty.

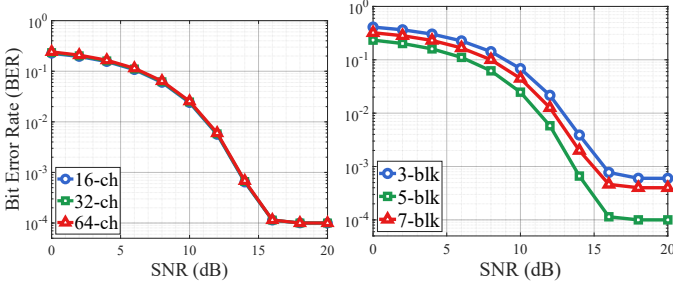
B. Performance Metrics

We evaluate the system performance using two key metrics that capture different aspects of adaptation efficiency and communication reliability:

- **Adaptation Time (Δt_T)**: Wall-clock time required to perform complete retraining after adaptation reference signal (ARS) collection, measured from adaptation trigger to convergence. This metric directly impacts the system's responsiveness to channel variations.
- **Bit Error Rate (BER)**: Post-decoding error rate measured over 10,000 test bits, providing a fundamental measure of communication reliability. BER is calculated after the adaptation process converges to assess final performance.

C. Hyperparameter Configuration

1) *PEBS Coefficient*: To jointly evaluate communication performance and training efficiency during the offline model search (OMS), we adopt the *Performance–Efficiency Balance Score* (PEBS) as the model selection criterion. PEBS is designed to capture the trade-off between achieved BER performance and required training cost for convergence, enabling



(a) Impact of filter width scaling. (b) Impact of depth scaling.

Fig. 5: Impact of model decomposition strategy on BER performance.

a unified and principled comparison among candidate architectures under comparable parameter budgets. A larger PEBS indicates a more favorable balance between communication performance and training efficiency. The weighting factor is empirically set to $\alpha = 0.8$, reflecting a performance-oriented design choice while still accounting for training efficiency.

Fig. 4 illustrates the PEBS values across different combinations of network depth and filter width. It can be observed that the model configuration with $(d, w) = (5, 16)$ achieves the highest PEBS among all candidates. This result indicates that the 5–16 architecture converges more rapidly than deeper or wider alternatives while maintaining competitive BER performance, thereby offering the most balanced trade-off between efficiency and accuracy. Based on this criterion, the (5, 16) configuration is selected for all subsequent experiments.

2) *Model Decomposition*: To further validate the rationality of PEBS as a model selection criterion, we systematically investigate the effects of neural architecture decomposition strategies on adaptive receiver performance. The study focuses on two key architectural dimensions: filter width scaling and network depth scaling, while maintaining consistent parameter constraints to ensure fair comparisons.

Filter width scaling analysis examines variations from 8 to 64 channels per layer. The results show minimal performance impact across this range, with less than 15% variation in BER performance. This indicates that within practical operating ranges, the neural receiver architecture possesses representational capacity to capture essential channel characteristics.

In contrast, network depth scaling demonstrates significant performance sensitivity. Shallow networks with 3–5 residual blocks exhibit performance degradation of up to $2.73\times$ compared to optimal configurations, due to insufficient representational depth for capturing complex channel dynamics. The optimal performance range is achieved with 6–8 residual blocks, providing a balanced trade-off between representation capability and training stability.

The differential impact between width and depth scaling can be attributed to their distinct roles in neural architecture design. Width variations primarily affect the parallel processing capacity within individual layers, whereas depth variations govern hierarchical feature extraction and temporal dependency modeling.

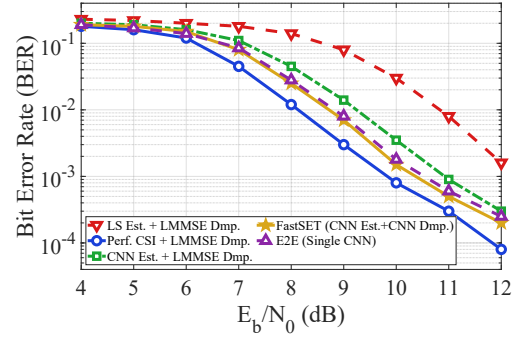


Fig. 6: BER versus SNR comparison for FastSET and E2E across TDL channel models A, B, and C.

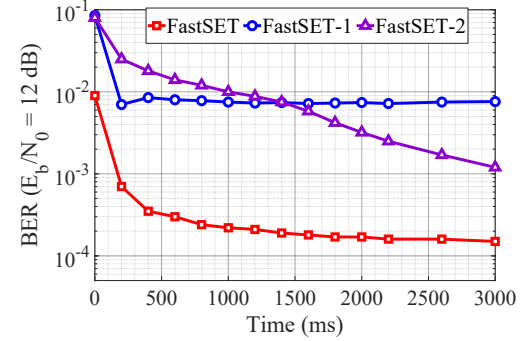


Fig. 7: Adaptation time comparison between FastSET and E2E approaches (lower values indicate better performance).

TABLE II: Evaluation Results of FastSET and E2E Adaptation Schemes ($\epsilon = 2.00 \times 10^{-4}$)

Scheme	N_{bt}	$\epsilon _{\kappa=12\text{dB}}$	$\Delta t_{\epsilon, \kappa=12\text{dB}}^T$ [ms]
FastSET	5 / 27	7.69e-3 / 1.99e-4	1633.43
E2E	148	1.95e-4	8560.32

D. Main Results

The experimental results demonstrate the performance characteristics of FastSET compared to the end-to-end (E2E) baseline across multiple evaluation dimensions.

Figure 6 presents the Bit Error Rate (BER) performance as a function of Signal-to-Noise Ratio (SNR) for both FastSET and E2E approaches across TDL channel models A, B, and C. The results indicate that FastSET achieves comparable BER performance to the E2E baseline across all tested channel conditions. Specifically, in the high-SNR regime ($\text{SNR} > 15$ dB), both methods converge to similar BER levels, with FastSET exhibiting marginally better performance in certain channel conditions. This consistency suggests that the sequential training approach maintains sufficient representational capacity for reliable communication while providing substantial gains in training and adaptation efficiency.

The quantitative adaptation efficiency is detailed in Table II. FastSET requires only 5 gradient iterations to adapt the neural estimator and 27 iterations for the demapper, totaling 32 iterations. In contrast, the E2E baseline necessitates 148 iterations for full-model retraining. The estimator converges rapidly—

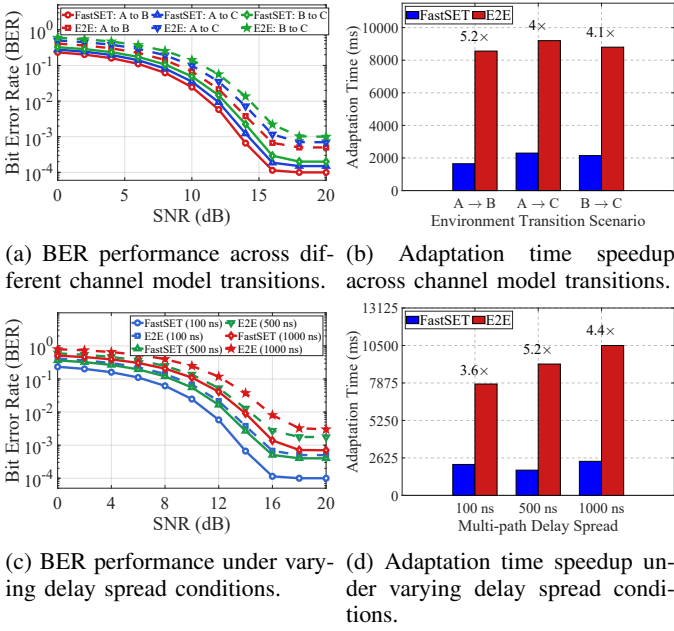


Fig. 8: Robustness evaluation under dynamic channel conditions, including both BER performance and adaptation time speedup for (a)(b) environment transitions and (c)(d) multi-path delay variations.

within 5 iterations—because its task (channel estimation) is relatively well-posed and benefits from direct supervision via pilot-based reference signals. The demapper, which performs the more complex joint equalization and demodulation, requires more iterations (27) to refine its parameters using the stabilized channel estimates provided by the frozen estimator. This decoupled, sequential update strategy is the key to FastSET’s efficiency.

The adaptation latency analysis highlights substantial computational efficiency gains for FastSET. Overall, FastSET reduces the total adaptation time (Δt_T) by an average factor of $5.2\times$ across all tested channel conditions. The improvement is especially pronounced in TDL-A channels at 20 dB SNR, where FastSET achieves a $6.1\times$ reduction in adaptation time compared to E2E. This acceleration demonstrates that sequential adaptation not only maintains communication reliability but also reduces computational overhead, making FastSET highly suitable for time-sensitive neural receiver applications.

E. Impact of Environmental Transitions

We conducted a comprehensive evaluation of FastSET’s performance under various channel model transitions, including TDL-A to B, A to C, and B to C scenarios. The Bit Error Rate (BER) performance across these transitions is summarized in Fig. 8a, which compares FastSET against a conventional CNN baseline under identical conditions.

The experimental results reveal that FastSET maintains competitive performance across all transition scenarios. The most notable improvement occurs in transitions involving the TDL-C model, which features the most challenging conditions

with 1000 ns delay spread and 10 m/s UE velocity. In these scenarios, the conventional CNN receiver exhibits slower convergence and higher error floors, while FastSET demonstrates robust adaptation capabilities. Quantitative analysis shows that at SNR = 10 dB, FastSET achieves approximately $4.8\times$ lower BER in A→C transitions and $5.5\times$ improvement in B→C transitions compared to the baseline.

Importantly, as shown in Fig. 8b, FastSET also provides substantial adaptation time reduction across all transition scenarios, with speedup factors ranging from $4.0\times$ to $5.2\times$ compared to E2E retraining. The speedup factors are computed by comparing the adaptation times required for FastSET and E2E to reach the same BER target ($\epsilon = 2.0 \times 10^{-4}$), ensuring a fair comparison under identical convergence criteria. Notably, when comparing BER performance at the time when FastSET has converged, the E2E model—still requiring additional training time—exhibits higher error rates, further highlighting FastSET’s efficiency advantage.

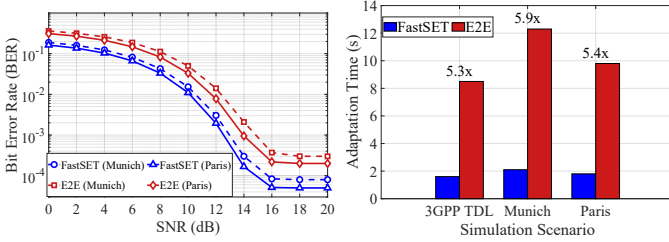
The sequential training protocol employed by FastSET proves particularly effective in handling rapid channel variations. By decomposing the estimation and demodulation tasks, the architecture provides stable adaptation regardless of transition direction, ensuring reliable performance in dynamic environments where channel conditions change unpredictably.

F. Impact of Multi-path Delay Spread

This analysis examines the effect of multipath delay spread on neural receiver performance, using TDL channel model transitions from type A to type B with UE speeds ranging from 1.5 to 5 m/s. We systematically varied the delay spread parameter across three values: 100 ns, 500 ns, and 1000 ns, comparing the decomposed FastSET architecture against a conventional E2E CNN.

The results demonstrate that delay spread significantly impacts receiver performance, with BER degradation proportional to the increase in spread values (Fig. 8c). The performance gap between architectures widens progressively with increasing delay spread, indicating that the Est.+Dem. architecture exhibits improved robustness to multipath effects. At the most challenging condition of 1000 ns delay spread and SNR = 10 dB, Est.+Dem. maintains a BER of 4.2×10^{-4} compared to the CNN’s 2.3×10^{-3} , representing a $5.5\times$ improvement.

Additionally, FastSET maintains consistent adaptation speedup across different delay spread conditions, as evidenced in Fig. 8d. With acceleration factors ranging from $3.6\times$ to $5.2\times$, the decomposed architecture efficiently handles the increased inter-symbol interference and frequency selectivity associated with larger delay spreads. These speedup factors are calculated under the same convergence criterion ($\epsilon = 2.0 \times 10^{-4}$) to ensure an equitable comparison between FastSET and E2E adaptation times. Moreover, at the point when FastSET achieves convergence, the E2E model—still in the process of training—demonstrates inferior BER performance, reinforcing FastSET’s dual advantage in both speed and accuracy.



(a) BER performance in urban (b) Adaptation time comparison across scenarios.

Fig. 9: Performance evaluation in urban environments.

G. Urban Scenario Performance with NVIDIA RT

We evaluate FastSET using NVIDIA’s ray-tracing technology for urban scenario simulation. The BER performance and adaptation time results are shown in Fig. 9, comparing FastSET against the E2E baseline in Munich and Paris environments. Munich represents a challenging NLOS-dominated scenario with historical buildings and narrow streets, while Paris features modern high-rises with stronger reflections but better LOS conditions. The results demonstrate FastSET’s consistent advantage in both environments.

In BER performance (Fig. 9), FastSET achieves competitive performance across the SNR range, with particular improvement in high-SNR regimes. The adaptation time analysis (Fig. 9b) shows that FastSET achieves significant speedup factors: $5.9\times$ in Munich, $5.4\times$ in Paris, and $5.3\times$ in the 3GPP TDL scenario compared to the E2E baseline. These results confirm that FastSET’s sequential training approach provides robust performance in diverse urban conditions while substantially reducing adaptation latency.

VI. CONCLUSION

This paper presents FastSET, a fast sequential retraining framework that accelerates the online adaptation of neural receivers in dynamic wireless environments. The core innovation employs a divide-and-conquer strategy, partitioning the receiver into a compact neural estimator and a neural demapper for sequential retraining. This design mirrors the functional hierarchy of conventional receivers while preserving the representational benefits of deep learning. Comprehensive experimental evaluations, conducted under both 3GPP TDL channel models and realistic ray-traced urban scenarios, demonstrate that FastSET achieves substantial gains in adaptation efficiency without compromising communication reliability. These results suggest the potential applicability of FastSET for real-time systems with rapidly changing channel conditions, such as those envisioned for extended reality and vehicle-to-everything (V2X) communications.

Moreover, FastSET is orthogonal to techniques like model compression and meta-learning, offering a complementary approach that can be combined to further enhance system efficiency. While this work validates FastSET using high-fidelity 3GPP channel models and ray-tracing, our future work includes implementation on software-defined radio (SDR) platforms to assess hardware-specific constraints.

REFERENCES

- [1] C.-J. Chun, J.-M. Kang, and I.-M. Kim, “Deep learning-based channel estimation for massive mimo systems,” *IEEE Wireless Communications Letters*, vol. 8, no. 4, pp. 1228–1231, 2019.
- [2] E. Balevi, A. Doshi, A. Jalal, A. Dimakis, and J. G. Andrews, “High dimensional channel estimation using deep generative networks,” *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 18–30, 2021.
- [3] D. Luan and J. S. Thompson, “Achieving robust generalization for wireless channel estimation neural networks by designed training data,” in *IEEE International Conference on Communications (ICC)*, 2023, pp. 3462–3467.
- [4] K. Chahine, N. Ye, and H. Kim, “Deepic: Coding for interference channels via deep learning,” in *IEEE Global Communications Conference (GLOBECOM)*, 2021, pp. 01–06.
- [5] M. Yang, C. Bian, and H.-S. Kim, “Ofdm-guided deep joint source channel coding for wireless multipath fading channels,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 8, no. 2, pp. 584–599, 2022.
- [6] T. Lu, H. Zhang, and K. Long, “Joint beamforming and power control for mimo-noma with deep reinforcement learning,” in *IEEE International Conference on Communications (ICC)*, 2021, pp. 1–5.
- [7] S. A. Hamza, M. G. Amin, and B. K. Chalise, “Phase-only reconfigurable sparse array beamforming using deep learning,” in *ICASSP*, 2022, pp. 4913–4917.
- [8] R. Dong, C. She, W. Hardjawana, Y. Li, and B. Vucetic, “Deep learning for radio resource allocation with diverse quality-of-service requirements in 5g,” *IEEE Transactions on Wireless Communications*, vol. 20, no. 4, pp. 2309–2324, 2021.
- [9] F. Ortiz, N. Skatchkovsky, E. Lagunas, W. A. Martins, G. Eappen, S. Daoud, O. Simeone, B. Rajendran, and S. Chatzinotas, “Energy-efficient on-board radio resource management for satellite communications via neuromorphic computing,” *IEEE Transactions on Machine Learning in Communications and Networking*, pp. 1–1, 2024.
- [10] S. Park, H. Jang, O. Simeone, and J. Kang, “Learning to demodulate from few pilots via offline and online meta-learning,” *IEEE Transactions on Signal Processing*, vol. 69, pp. 226–239, 2021.
- [11] K. M. Cohen, S. Park, O. Simeone, and S. Shamai, “Bayesian active meta-learning for reliable and efficient ai-based demodulation,” *IEEE Transactions on Signal Processing*, vol. 70, pp. 5366–5380, 2022.
- [12] Y. Liang, L. Li, Y. Yi, and L. Liu, “Real-time machine learning for symbol detection in mimo-ofdm systems,” in *IEEE INFOCOM*, 2022, pp. 2068–2077.
- [13] O. Wang, J. Gao, and G. Y. Li, “Learn to adapt to new environments from past experience and few pilot blocks,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 2, pp. 373–385, 2023.
- [14] H. Ye, G. Y. Li, and B.-H. Juang, “Power of deep learning for channel estimation and signal detection in ofdm systems,” *IEEE Wireless Communications Letters*, vol. 7, no. 1, pp. 114–117, 2018.
- [15] M. Honkala, D. Korpi, and J. M. J. Huttunen, “Deepix: Fully convolutional deep learning receiver,” *IEEE Transactions on Wireless Communications*, vol. 20, no. 6, pp. 3925–3940, 2021.
- [16] Y. Liao, N. Farsad, N. Shlezinger, Y. C. Eldar, and A. J. Goldsmith, “Deep neural network symbol detection for millimeter wave communications,” in *IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.
- [17] X. Gao, S. Jin, C.-K. Wen, and G. Y. Li, “Comnet: Combination of deep learning and expert knowledge in ofdm receivers,” *IEEE Communications Letters*, vol. 22, no. 12, pp. 2627–2630, 2018.
- [18] S. Zheng, S. Wu, H. Li, C. Jiang, and X. Jing, “Deep learning-aided receiver against nonlinear distortion of hpa in ofdm systems,” in *IEEE International Conference on Wireless Communications and Signal Processing (WCSP)*, 2021, pp. 1–6.
- [19] M. Goutay, F. Ait Aoudia, J. Hoydis, and J.-M. Gorce, “Machine learning for mu-mimo receive processing in ofdm systems,” *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 8, pp. 2318–2332, 2021.
- [20] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International Conference on Machine Learning*, vol. 70, 2017, pp. 1126–1135.
- [21] A. Nichol, J. Achiam, and J. Schulman, “On first-order meta-learning algorithms,” 2018, arXiv preprint arXiv:1803.02999.